AA1 - Investigación documental y especificación de requerimientos

Grupo 02 (Tercer Ciclo):
Darwin Campoverde
Javier Guarnizo
Ricardo Ochoa
Diyer Torres
Mateo Yanangomez

Carrera de Ingeniería en Ciencias de la Computación

7 de octubre de 2026

1. Introducción

Como estudiantes de **tercer ciclo** de la carrera de Ingeniería en Ciencias de la Computación, abordamos la propuesta del proyecto enfocado en el **Triaje Automatizado y Priorización en Urgencias del Hospital Isidro Ayora** [1]. La necesidad de este proyecto surge de los cuellos de botella e interrupciones dinámicas en la atención hospitalaria, donde la variabilidad en los signos vitales de los pacientes exige un sistema capaz de recalcular continuamente el orden de atención según la escala de severidad médica [2]. El prototipo se concibe como una aplicación desarrollada sobre **Node.js** e interfaz en **React** con **Tailwind CSS**, adaptando el marco del Índice de Severidad de Emergencia (ESI) mediante algoritmos de priorización dinámica [3].

El núcleo conceptual del sistema se sostiene sobre dos pilares fundamentales: la **Gestión Eficiente de Triaje Médico** y el **Análisis Estructural de Memoria en el Motor V8** [3]. Desde la perspectiva biomédica y algorítmica, el desarrollo no depende de librerías nativas auxiliares para el manejo de la cola; en su lugar, requiere la implementación artesanal de estructuras de datos hechas a medida: una **Cola de Prioridad Máxima (Max-Heap)** para la reorganización en tiempo real y una **Lista Enlazada Ordenada** para el seguimiento cronológico y gestión de historiales de pacientes [4].

Para garantizar que el sistema procese actualizaciones constantes de signos vitales sin colapsar el entorno de ejecución ni comprometer la fluidez de la interfaz web, es indispensable comprender la forma en que el motor V8 administra los recursos físicos [5]. V8 distribuye la información en dos zonas fundamentales:

1. **La Pila de Llamadas (Call Stack):** Una región de memoria rápida y de tamaño acotado ($\approx$ 1 MB en Node.js) que almacena el contexto de ejecución de las funciones, marcos de llamada, punteros y valores primitivos [5]. En este proyecto, las variables de cálculo ESI (`number`), estados de alerta (`boolean`) y métodos de reordenamiento estático se procesan en la pila. La ejecución recursiva de reordenamiento dinámico (`heapifyUp` y `heapifyDown`) debe controlarse cuidadosamente para evitar desbordamientos (`RangeError: Maximum call stack size exceeded`).
2. **El Montón de Memoria (Memory Heap):** Una región dinámica y extensa donde se instancian los objetos de tipo paciente, las estructuras del arreglo que simulan el *Max-Heap* y los nodos de la *Lista Enlazada Ordenada* [3]. Esta zona está supervisada por el Recolector de Basura (*Garbage Collector*), el cual libera la memoria de los pacientes atendidos únicamente cuando sus nodos rompen todas las referencias internas [3].

En consecuencia, el prototipo no solo cumple el propósito de optimizar la priorización en el área de urgencias del Hospital Isidro Ayora, sino que constituye una plataforma experimental para validar la eficiencia temporal de reordenamiento ($O(\log n)$) y la liberación efectiva de espacio en el montón de V8 [2].

2. Figuras y Tablas

Para formalizar la propuesta del sistema bajo estándares de ingeniería de software, en la Tabla 1 se detallan los Requerimientos Funcionales (RF) y No Funcionales (RNF) definidos según la norma IEEE 830 / ISO 29148 abreviada [1].

Tabla 1: Especificación de Requerimientos del Sistema de Triaje bajo norma IEEE [1].

Tipo	Código	Descripción del Requerimiento / Criterio de Aceptación
RF	RF-01	Ingreso y registro de pacientes asignando nivel de severidad ESI e id único.
RF	RF-02	Implementación nativa de un **Max-Heap** con métodos `heapifyUp` y `heapifyDown`.
RF	RF-03	Recálculo dinámico de prioridad ante la modificación de signos vitales del paciente.
RF	RF-04	Extracción del paciente más grave ($O(1)$) y transferencia a la **Lista Enlazada Ordenada**.
RF	RF-05	Tablero visual en React con alertas de tiempo promedio de espera y re-categorización.
RNF	RNF-01	Mantenimiento del `heapUsed` eficiente evitando *memory leaks* en objetos paciente [2].
RNF	RNF-02	Complejidad logarítmica $O(\log n)$ en las operaciones de reordenamiento del Heap [4].
RNF	RNF-03	Eliminación explícita de referencias (`node = null`) tras desencolar para el *Garbage Collector* [3].

A nivel de arquitectura interna en JavaScript (Node.js/React), la Tabla 2 muestra la matriz de trazabilidad entre los componentes del proyecto de triaje, las estructuras diseñadas y su ubicación física en el motor V8 [5].

Tabla 2: Ubicación en Memoria V8 y Complejidad de las Estructuras del Sistema de Triaje [5].

Módulo del Sistema	**Estructura Utilizada**	**Ubicación V8**	**Complejidad**
Nivel ESI y Edad	Primitivos (`number`)	Call Stack	Acceso $O(1)$
Cédula y Nombre Paciente	Cadenas (`string`)	Memory Heap	Comparación $O(n)$
Cola de Espera Urgencias	Max-Heap (Arreglo)	Memory Heap	Reorganizar $O(\log n)$
Historial de Atendidos	Lista Enlazada Ordenada	Memory Heap	Inserción $O(n)$
Estado de Alertas	Booleans e Identificadores	Call Stack	Acceso $O(1)$
Ejecución `heapify`	Marcos de Función	Call Stack	Límite ≈ 1 MB

Para evaluar el comportamiento del montón en Node.js al manipular las estructuras del sistema de triaje, se diseñó la función del Código 1, que simula el uso de memoria en V8 en tiempo de ejecución [2].

```
# Medicion de memoria en el motor V8 para el sistema de Triaje
def evaluar_consumo_triaje(num_pacientes):
  # Simulacion del impacto en memoria por paciente en el Heap
  print(f"Hospital Isidro Ayora: Procesando {num_pacientes} pacientes...")
  heap_usado_mb = (num_pacientes * 64) / (1024 * 1024)
  print(f"Memoria Heap estimacion V8: {heap_usado_mb:.2f} MB")
```

Código 1: Monitoreo del Heap en Node.js para el Sistema de Triaje Hospitalario [2].

3. Conclusiones

- La propuesta del proyecto de Triaje Automatizado para el Hospital Isidro Ayora demuestra que la aplicación de una cola de prioridad mediante un **Max-Heap** alineado con la arquitectura de V8 optimiza la respuesta del sistema ante emergencias [5]. Mantener la re-categorización en $O(\log n)$ evita bloqueos en la *Call Stack*, permitiendo que la interfaz desarrollada en React y Tailwind CSS funcione fluidamente en tiempo real [4].
- El cumplimiento estricto de los requerimientos no funcionales (RNF-01 y RNF-03) garantiza una gestión de memoria sustentable y libre de fugas (*memory leaks*) [2]. La desvinculación explícita de punteros en la **Lista Enlazada Ordenada** y en las extracciones del Heap permite al *Garbage Collector* operar óptimamente, asegurando la estabilidad del sistema durante flujos intensos de pacientes en urgencias [3].

4. Bibliografía

[1] ISO/IEC/IEEE, «Systems and software engineering — Life cycle processes — Requirements engineering,» International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical y Electronics Engineers, Geneva, Switzerland, inf. téc. ISO/IEC/IEEE 29148:2018, 2018.

[2] Node.js Contributors. «Process: process.memoryUsage(),» visitado 5 de oct. de 2026. dirección: `https://nodejs.org/api/process.html#processmemoryusage`

[3] V8 Team. «Trash talk: the Orinoco garbage collector,» visitado 5 de oct. de 2026. dirección: `https://v8.dev/blog/trash-talk`

[4] S. Guardati, *Estructuras de datos básicas: programación orientada a objetos con Java*. México: Alpha Editorial, 2015.

[5] Mozilla Developer Network. «Memory management in JavaScript,» visitado 5 de oct. de 2026. dirección: `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Memory_management`